Budgeted Incident Detection: an On-Call Queue That Shows the Right K Items a Day

Course: Software Engineering and Programming **Team member:** Aishah Mohamed Alyammahi

Motivation

Teams that run online platforms, shops or sensor networks receive thousands of alerts and notifications a day, and the person on duty can read only a few. Real problems are missed because attention runs out, not because monitoring is silent. The usual rule — escalate by log severity and delete exact duplicates — only reads logs. On the public GAIA microservice benchmark, **82% of injected faults never write a log line**; they show up only as a jump in a metric such as memory. The problem is therefore a *budget* problem: given at most K items a day, which ones should a person see, as they happen?

Expected impact

- **On-call engineers and small operations teams** get one short daily list (about 18 items instead of hundreds of raw alerts) that catches more real problems with fewer false items.
- **Any team with a metric over time** — orders per minute, sensor readings, ad clicks, notification volume — can check, without code, whether the tool would have caught its own past problems before trusting it.
- **Students and researchers** get a reproducible, honestly evaluated benchmark: fair baselines, a real-time rule, and a random-chance line.

Proposed approach

A web application and Python library that builds a **causal daily queue** of at most K items, first come first served:

1. **Metric jumps** — a point far above its own recent median (0.5 GB for memory, or a unit-free robust score for any metric), confirmed after two points (≈ 0.7 min).
2. **Slow leaks** (3-hour window) and **monitoring gaps** (a series that stopped sending data).
3. **Log alerts and notifications**, ranked by severity or by a cost-sensitive classifier (a miss costs 10× a false alarm), limited to a quarter of the slots.

Main features: an *Evaluate my data* page (upload any CSV; columns detected automatically; the app tunes on the early part and reports results on the rest); one-click use of the tuned settings; shadow mode with Real/Noise marks for retraining; Alertmanager, Prometheus and PagerDuty connectors; a command-line tool.

Technologies: Python, pandas, scikit-learn, FastAPI, SQLite, HTML/JavaScript, Docker, pytest, Git. **Data:** GAIA MicroSS (CloudWise), the Numenta Anomaly Benchmark (MIT licence), and a synthetic online-shop dataset.

Deliverables and success criteria

Deliverables: the working web app and CLI; the source code with automated tests; experiment scripts that regenerate every result; a final report, user guide and demo.

Criterion	How it is measured	Target
Beats the rules teams already run	Faults caught at K = 20 vs log severity, a static threshold and a Prometheus-style rate rule, same queue	Higher than all three, with fewer false items
Not a lucky period	Walk-forward over 8 later weeks, everything re-learned from the past	Wins most weeks; mean and 95% interval reported
Real time	No item chosen with information from later in the day	Confirmed within about a minute
Works beyond one dataset	NAB (5 domains) and the shop data, each against a chance line	Above chance, with failures reported

Criterion	How it is measured	Target
Usable by others	A new user evaluates their own CSV without code	End-to-end in the app; covered by tests
Reproducible	`run_validate.py` fails if the report and the results disagree	Passes on a clean run

Timeline and feasibility

Weeks	Milestone
1–2	Data pipeline, GAIA loading and clock alignment, baseline rule
3–4	Alert classifier; metric-jump, leak and gap detectors
5–6	Causal budget queue; fair baselines; walk-forward and statistics
7	Web app, connectors, *Evaluate my data*, CLI
8	Robustness tests, NAB evaluation, report, demo

Key risks and mitigations.

- *Narrow benchmark:* every GAIA fault is the same 1 GB memory program → add synthetic fault shapes and the NAB benchmark, and state the limit.
- *Straw-man baseline:* log severity catches 0% → compare with real metric rules and report the modest margin honestly.
- *No labels at a new company:* rules-only mode on day one; Real/Noise marks become labels later.
- *Monitoring outages:* shown as gap items, because no detector can see a fault without data.

Dependencies: the public GAIA and NAB datasets (downloaded, not stored in Git) and a standard Python 3.10–3.12 environment; no paid services are required.